

تعدد الأشكال

Polymorphisme

تمهيد

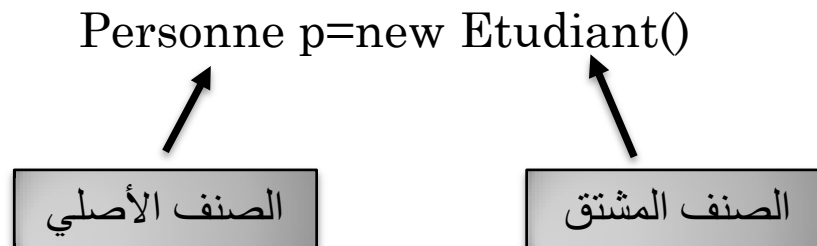
- من المفاهيم الأساسية في البرمجة الكائنية يكمل الوراثة
- تعدد الأشكال يكون باستعمال الاشتقاق, اعادة التعريف و التعريف المتعدد
- يمكن للكائن أن يأخذ أشكال مختلفة أثناء تطبيق البرنامج
- التعامل مع الكائنات بدون المعرفة الدقيقة لنوعهم



المبادئ الأساسية

Class **Etudiant** extends **Personne**{...}

```
Personne p=new Personne()  
p=new Etudiant()
```



- يمكن اسناد لمتغير معرف في صنف ما صنف مشتق منه



الاشتقاق المتتالي

```
class A{
```

```
class B extends A
```

```
class C extends A
```

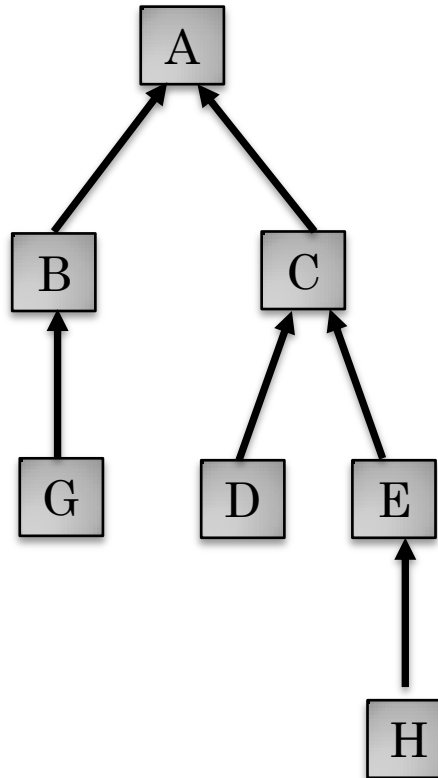
```
class D extends C {...}
```

```
class E extends C{...}
```

```
class G extends B{...}
```

```
class H extends E{...}
```

أمثلة لاسنادات



```
G g; C c; D d;  
A a = new A();  
a = g; // صحيح  
a = d; // صحيح  
d = a; // خطأ  
c = d; // صحيح  
c = g; // خطأ
```



المبادئ الأساسية

Class **Etudiant** extends **Personne**{...}

Class **Enseignant** extends **Personne**{...}

```
Personne p=new Personne()
```

```
p.afficher()           // Personne
```

```
p=new Etudiant()
```

```
p.afficher()           // Etudiant
```

```
p=new Enseignant()
```

```
p.afficher()           // Enseignant
```

الدالة afficher يجب أن تكون موجودة
في **Personne**
لا يمكن كتابة `p.getMat()` مثلا

- الطرق المستعملة تتبع الصنف المستعمل.
- اختيار الطريقة المستعملة يكون عند التطبيق (المعلومة غير معروفة عند التصريف)
- ما يسمى الارتباط الديناميكي للطرق (liaison dynamique)



المبادئ الأساسية

استعمال جدول

```
Personne[ ] pers={new Etudiant(), new Enseignant() };  
For (Personne p:pers)  
{  
    p.afficher;           // pers[0].afficher Etudiant  
                           // pers[1].afficher Enseignant  
}
```



اعادة صياغة الطرق

```
class A{...  
    private void f()  
...  
}
```

```
class B extends A
```

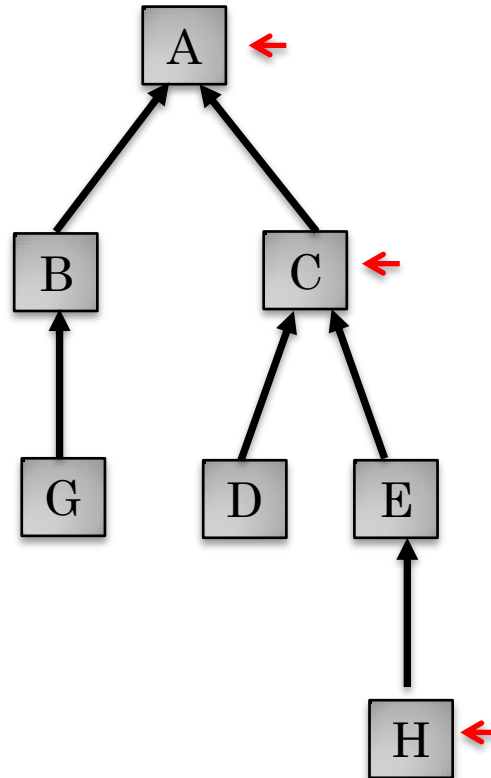
```
class C extends A{...  
    private void f()  
...  
}
```

```
class D extends C  
{...}
```

```
class E extends D  
{...}
```

```
class G extends B{...}
```

```
class H extends E{...  
    private void f()  
...  
}
```



```
a=g;  
a.f();    // f--> A
```

```
a=c;  
a.f();    // f--> C
```

```
e=h;  
e.f();    // f--> H
```



إعادة صياغة الطرق و التعريف المتعدد

```
class A{...  
    private void f(double x)  
    ...  
}
```

```
class B extends A{...  
    private void f(float x)  
    private void f(double x)  
    ...  
}
```

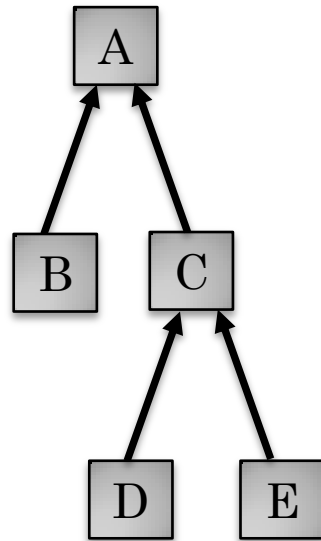
```
class C extends B{...  
    private void f(double x)  
    ...  
}
```

```
A a=new A();  
B b=new B();  
C c=new C();  
float x; double y;  
a.f(x);           // f(double)--> A  
a.f(y);           // f(double)--> A  
b.f(x);           //f(float) --> B  
a=b;  
a.f(x);           // f(double)--> B  
b=c;  
b.f(x);           // f(float)--> B
```

اختيار انسب طريقة (امضاء) يكون عند التصريف و تبقى ثابتة



حالة الكائن معامل لدالة



```
static void f(A a){  
    A a1;  
    C c;  
    E e;  
    a1=a; // صحيح  
    c=a;   // خطأ  
    e=a;   // خطأ  
}
```

```
a= new A();  
c= new C();  
e= new E();  
f(a); // صحيح  
f(c); // صحيح  
f(e); // صحيح
```

- عند مناداة الدالة المعامل يطبق قاعدة الاسناد فالحالات الثلاثة صحيحة .
- داخل الدالة الأمر يختلف حيث لا يمكن للمتصرف معرفة نوع الكائن بدقة فيقبل الحالة الأولى فقط.

اجبار تغيير النوع

Le CAST, le transtypage

```
static void f(A a){  
    A a1;  
    C c;  
    E e;  
    a1=a; // محقق  
    c= (C) a;  
    e=(E) a;  
}
```

- استعمال طريقة تغيير النوع الصريحة (cast)
- التصريف يمر بطريقة صحيحة غير أنه سيكون مشكل في التطبيق في حالة اسناد خاطئ.



حالة الكائن معامل لدالة

التحقق من الصنف

instanceof

```
static void f(A a){  
    A a1;  
    C c;  
    E e;  
    a1=a;  
    if (a instanceof C)  
        {c= (C) a;}  
    if (a instanceof E)  
        {e=(E) a;}  
}
```

instanceof معامل ثنائي نتيجه منطقية
تكون صحيحة في حالة ما اذا كان a مثال لـ C أو واحد من أصوله

• استعمال instanceof للتأكد من الصنف قبل الاسناد

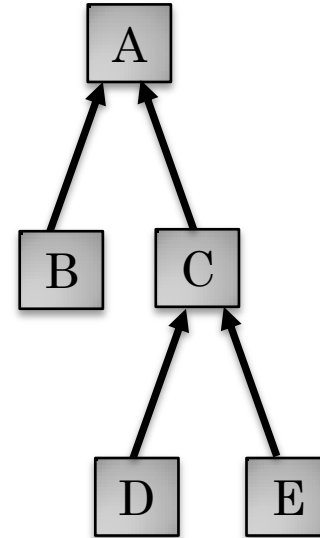
حالة الكائن معامل لدالة

مع تعدد للتعاريف

```
public class PP {  
    static void f(A a){  
        ...  
    }  
}
```

```
static void f(C c){  
    ...  
}
```

```
public static void main(String[] args) {  
    A a= new A();  
    C c= new C();  
    E e= new E(); B b=new B();  
    f(a);    // f(A)  
    f(c);    // f(C)  
    f(e);    // f(C)  
    f(b);}   // f(A)
```



الصف Object

- كل الأصناف تشتق من هذا الصف
- لا نحتاج لكتابة extends
- الطرق الموجودة فيه يمكن استعمالها في كل الأصناف

الطريقة	الاستعمال
hashCode	يمثل رقم وحيد لكل كائن تلقائيا يمثل hashCode لعنوان الكائن في الذاكرة
equals	لمقارنة عناوين كائنين (أمثلة لأصناف)
toString	تعطي اسم صنف كائن بالإضافة ل hashCode الكائن
getClass()	تعطي معلومات عن الكائن الحالي
finalize()	طريقة تستعمل قبل هدم الكائن ب GC

